Unit - I

Basic Syntactical Constructs in Java

(10 Marks)

Fundamental of Programming

Two types of Programing Paradigm

POP	**OOP**
-Linear Flow of data by functions	-Non Linear flow of data by Object
-Structure Oriented	-Object Oriented
-Top-down approach	-Bottom-up approach.
- e.g.: C	- e.g.: C++ & JAVA

Fundamental of Programming

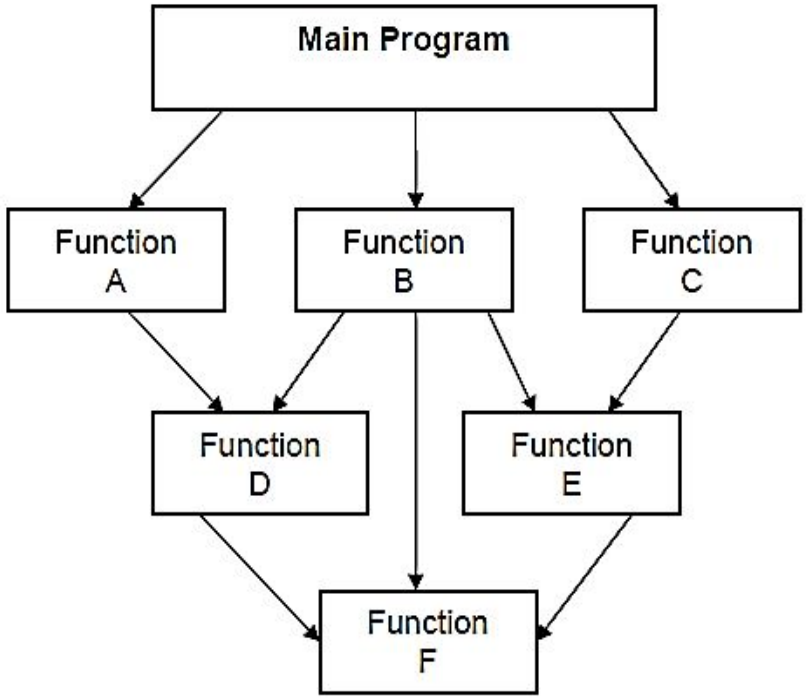


Fig 1.2 Procedure oriented programming Methodology

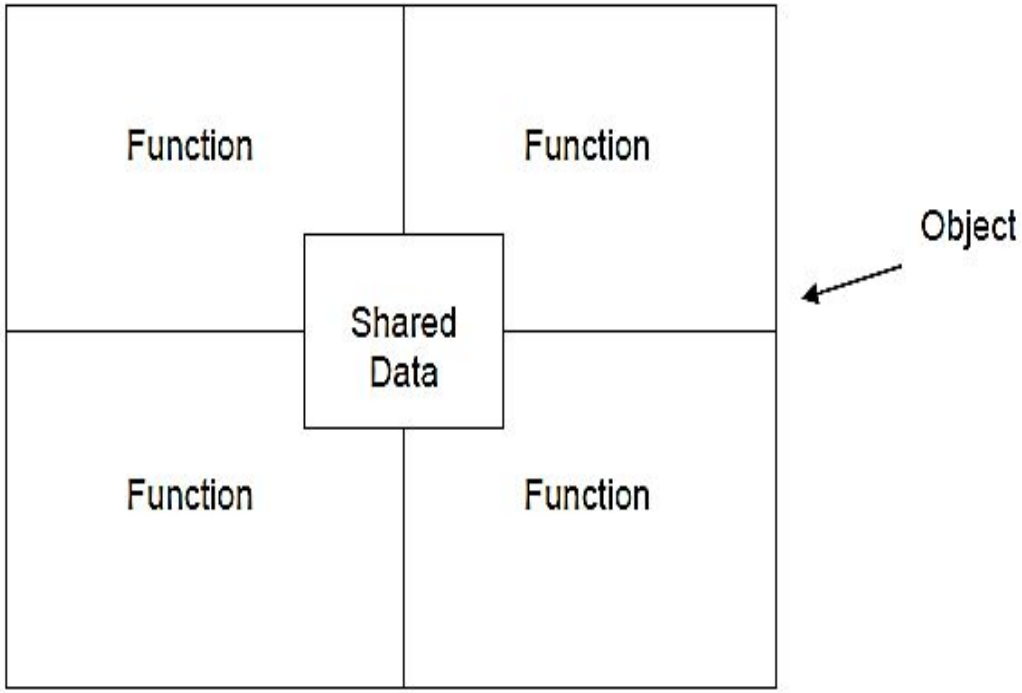


Fig 1.3 Object Oriented Methodology

What is Java???

- JAVA is general purpose (high level) programming language, designed to produce programs that will **run on any computer** system.

Or

- JAVA is programming language expressly designed for use in the distributed environment of the internet

- Java is an Object-Oriented programming language developed by **James Gosling** in the early 1990s.

History of Java

- In 1990, Sun Micro Systems Inc. (US) was decided to develop a project for consumer electronic devices that could be controlled by a remote. This project was called Stealth (secrete) Project but later its name was changed to Green Project.

- In January 1991, Project Manager James Gosling and his team members Patrick Naughton ,Mike Sheridan, Chris Wrath, and Ed Frank met to discuss about this project.

- Gosling thought C and C++ would be used to develop the project. But the problem he faced with them is that they were system dependent languages.

History of Java

- James Gosling with his team started developing a new language, which was completely system independent.

- This language was initially called OAK. Since this name was registered by some other company, later it change to the name JAVA.

- James Gosling and his team members were consuming a lot of coffee while developing this language. Good quality of coffee was supplied from a place called “Java Island” at Indonesia. Hence they fixed the name of the language as Java. The symbol for Java language is cup and saucer.

- Sun microsystem formally announced Java at Sun World conference in 1995. On January 23rd 1996, JDK1.0 version was released

More about Java???

- JAVA is **PURE Object Oriented language**.
- **Two stage** system **Compiled** & **interpreter**
    - Java Compiler translate source code into byte code
    - Java interpreter translate byte code into machine code
    - Java compiler & interpreter together called as JVM(Java virtual machine)
- Write once run anywhere (WORA) strategies
    - Compiled java programs once can run on all platform that support java without recompilation.

Write Once's Run Anywhere

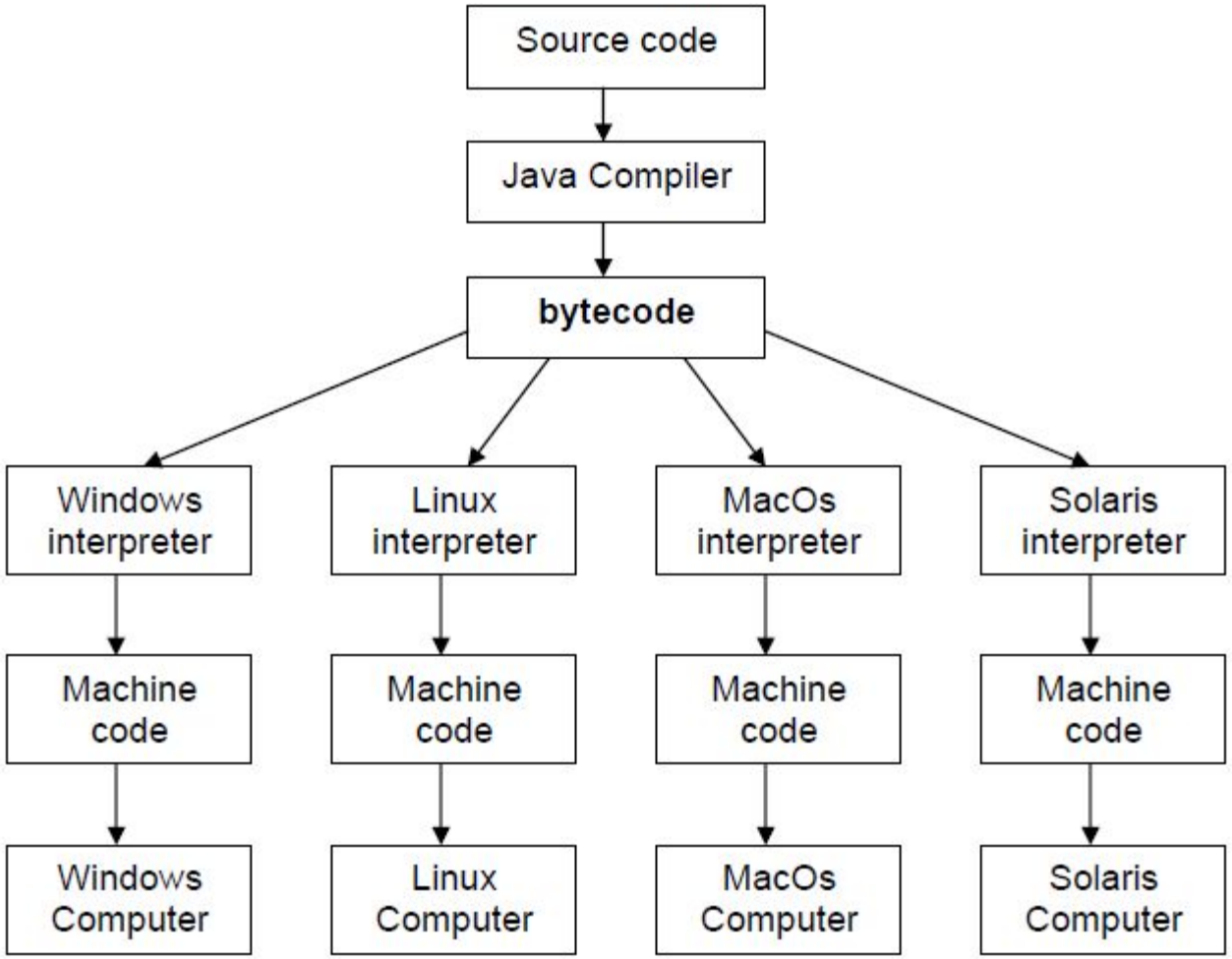


Fig. 1.11 Creation of the machine independent code

JAVA contain OOP Concept

- **Class:** set of object which further define set of Functions
- **Object:** Instance of class, which defines set of function
- **Data Abstraction:** Representing essential features without including the background particulars or clarification.
- **Data Encapsulation:** packaging up of data and methods(Functions) into the single unit.
- **Data Hiding:** The methods(Functions) which are packaged
- around the object can access the data.
- **Polymorphism:** Ability to take more than one form
- **Inheritance:** Adopting properties of Old class to new class
- **Binding:** (Static/dynamic)- Compiler knows the methods (values) before / After execution.

JAVA & C

- Java does not include unique statement like ‘sizeof’ & ‘typedef’
- Java doesn’t support ‘union’ & ‘structure’
- Java does not support ‘pointers’
- Java does not have preprocessor like ‘#define, #include etc.’
- Java adds new operators such as ‘instanceOf’ and >>>
- Java adds labeled break and continue

JAVA & C++

- Java does not support Operator Overloading
- Java does not have template class as in C++
- Java does not support multiple inheritance of classes, this is accomplished by new feature ‘interface’
- Java does not use pointers.
- Java replaced function like destructor() with function finalize().
- *In java we don’t use any header file.*

Features of JAVA

- Simple & Small: Java is designed to be easy for the professional programmer to learn and use.

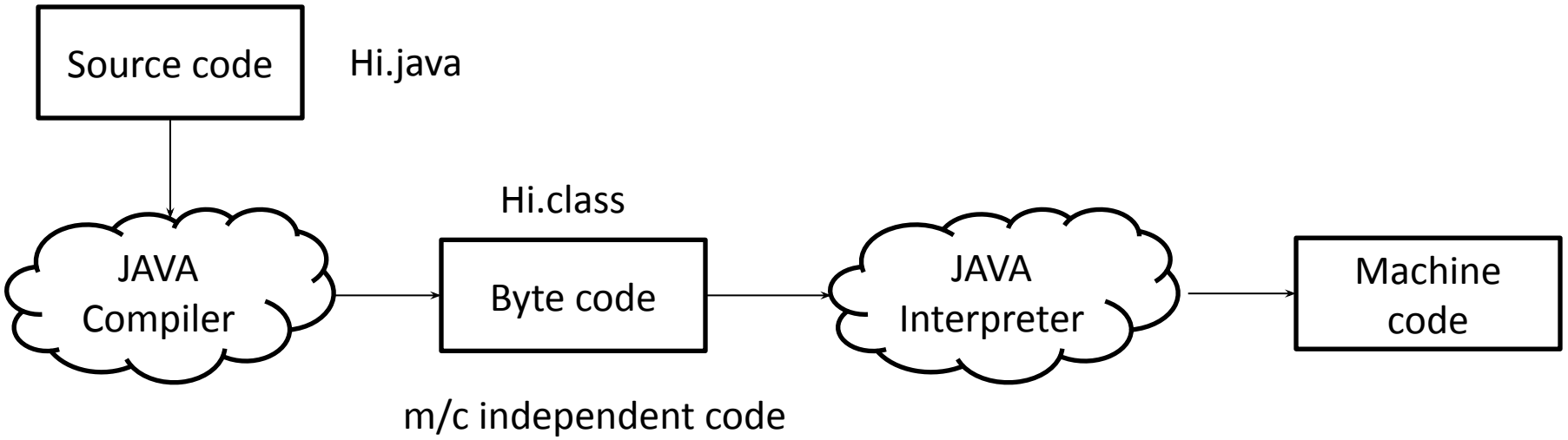
- object-oriented: Almost everything in java is an Object. All program code reside in classes. Java comes with set of classes arranged in packages.

- Compiled & Interpreter: Java compiler translates source code into byte code instruction. Byte code are not machine instruction. Java Interpreter translate byte code into machine code. Therefore JAVA also called Two-Stage system.(JVM)

- Robust & Secure: Restricts the programmer to find the mistakes early, performs compile-time (strong typing) and run-time (exception-handling) checks, manages memory automatically.

Features of JAVA

- Multithreaded: supports multi-threaded programming (multiple task run simultaneously w/o worrying about application)
- Platform Independent & Architecture-neutral: Java Virtual Machine provides a platform independent environment for the execution of Java byte code(WORA) write one's run anywhere Eg:



Features of JAVA

- Dynamic: Java is dynamic language support dynamic linking of libraries, class, methods, object etc. It also support functions of other language like C & C++.
- Distributed: Java also useful for Network, Multiple programmer at multiple remote location can share same project. Web browser support for java applet
- High-performance: Speed of execution of Java program is Fast as compared to C/C++. Due to two stage system.(compile + interpreter=JVM)
    - JVM=Java Virtual Machine

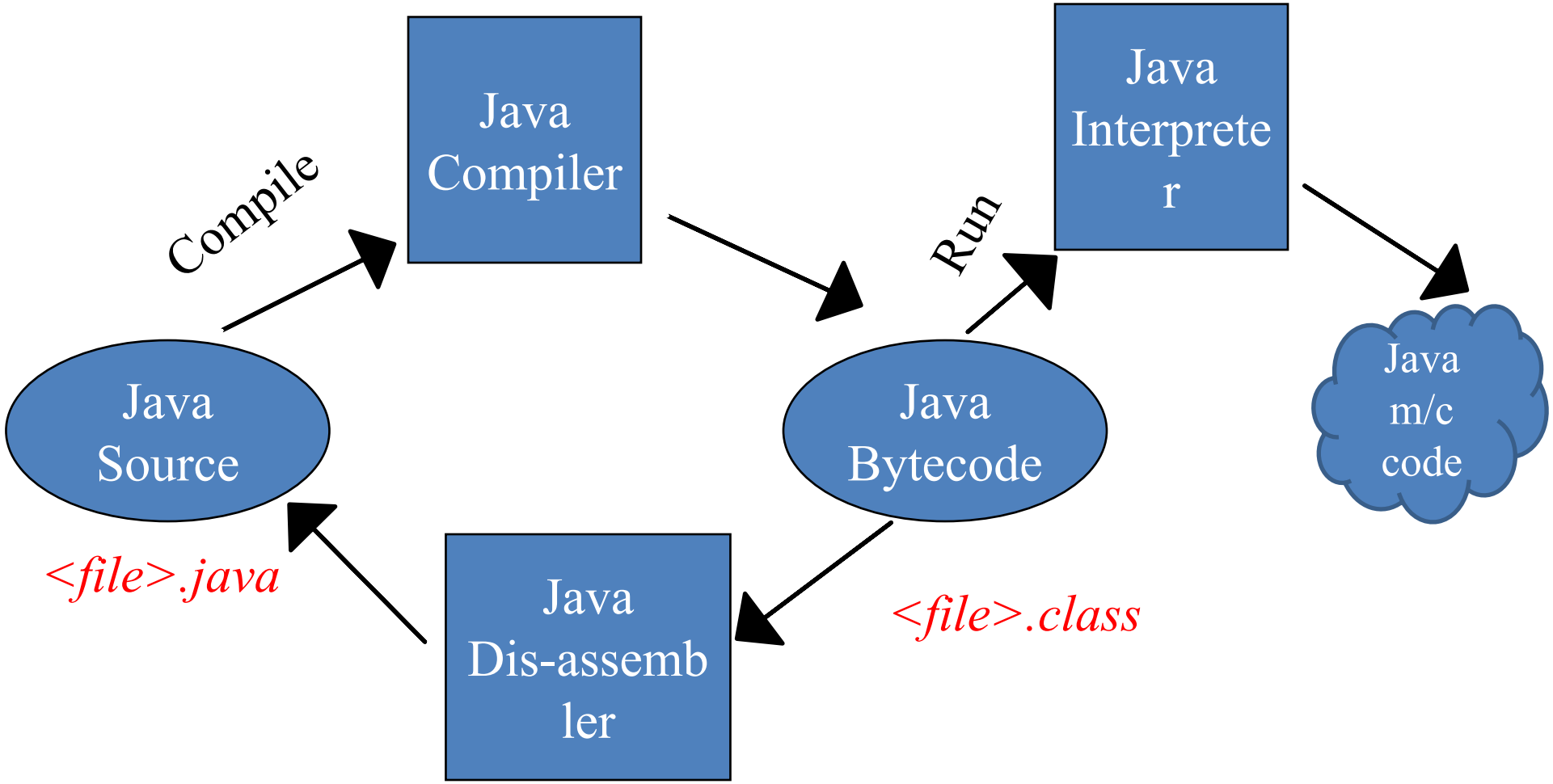
Java Environment

- Java environment includes large no. of development tools and hundreds of classes and functions (method in java).
- The development tools are part of system, known as JDK(Java Development Kit), Netbeans, Eclipse etc.
- And classes and methods are part of JSL(Java Standard Libraries)

Java Environment

- Java Development Kit JDK, comes under variety of tools use for developing & running java programs.
- **javac** (Java Compiler):
    - To translate source code into byte code
- **java** (Java Interpreter)
    - To translate byte code into machine code
- **javap** (Java disassembler)
    - To convert byte code file into program description.
- **javah** (For C Header file)
    - Produce header file for use with C program
- **javadoc** (for creating HTML documents)
    - Create HTML format documentation from java source code file
- **appletviewer** (for viewing java applet on browser)
    - To run java applets (without using browser)

Prepare and Execute Java



Java software development kit [SDK]

- Java SDK comes in three Editions

1) J2ME- Micro edition (for handheld & portable device)

2) J2SE- Standard edition (for PC development)

3) J2EE- Enterprise edition (Distributed & enterprise computing)

VERSIONS of Java-SE

- JDK 1.0 (jan 21, 1996)
- JDK 1.1 (feb 19, 1997)
- J2SE 1.2 (dec 8, 1998)
- J2SE 1.3 (may 8, 2000)
- J2SE 1.4 (feb 6, 2002)
- J2SE 5.0 (sep 30, 2004)
- Java SE 6 (dec 11, 2006)
- Java SE 7 (jul 28, 2011)
- Java SE 8 (mar 18, 2014)

Data Types in java

- *Every variable declared in a Java program is having the data type.*
- *Data type specifies the size and type of values that a variable can hold.*

Data Types

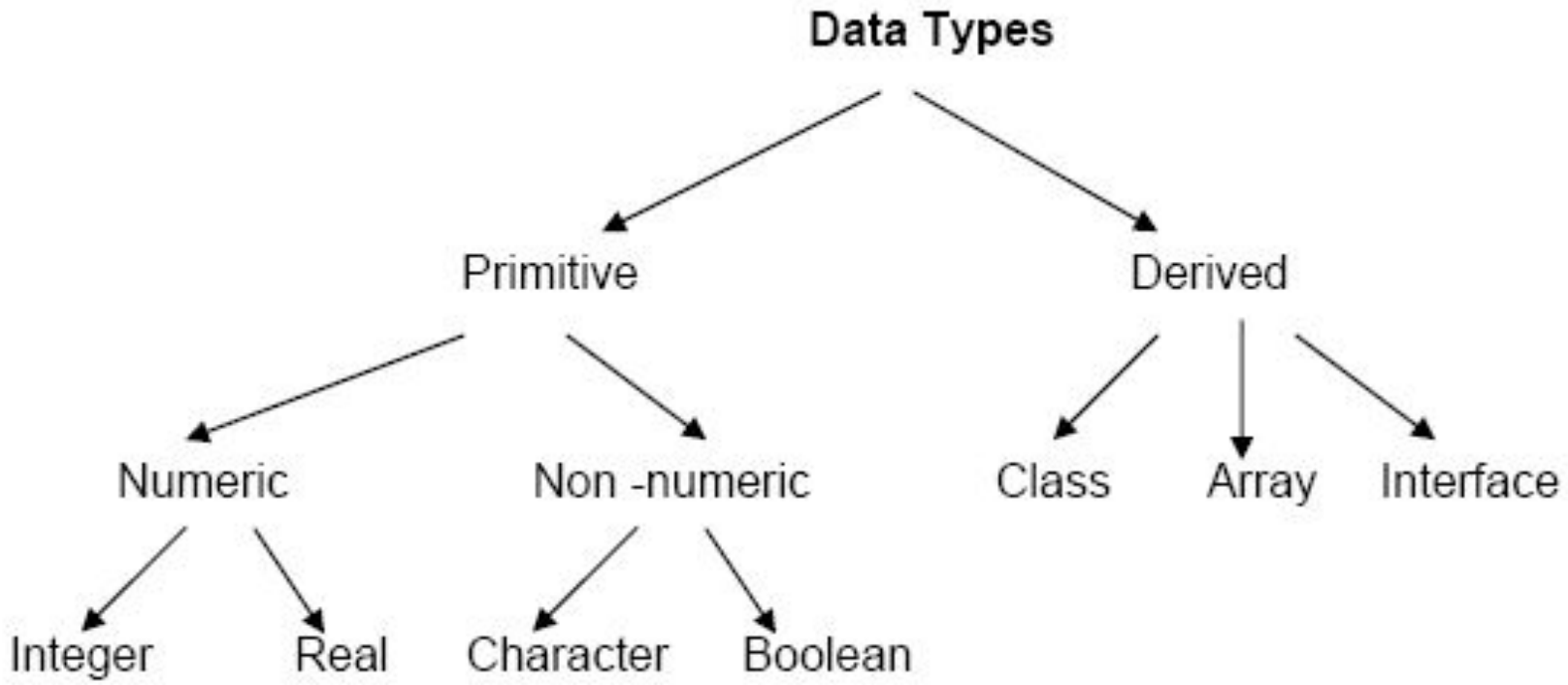


Fig. 1.18 Data Types in Java

Data Types in java

Data Type	Characteristics	Size in Bytes	Range
`byte`	8 bit signed integer	1 byte	-128 to 127
`short`	16 bit signed integer	2 byte	-32768 to 32767
`int`	32 bit signed integer	4 byte	-2,147,483,648 to 2,147,483,647
`long`	64 bit signed integer	8 byte	-9,223,372,036,854,775,808 to-9,223,372,036,854,775,807
`float`	32 bit floating point number	4 byte	± 1.4E-45 to ± 3.4028235E+38
`double`	64 bit floating point number	8 byte	± 4.9E-324 to ± 1.7976931348623157E+308
`boolean`	`true` or `false`	1 bit	NA
`char`	16 bit, Unicode	2 byte	`Any symbol, digit etc.`

Variables in java

- A variable is an identifier (entity) that denotes a storage location used to store the data value.
- Syntax:
  - <datatype> <name_of_variable>;
  - Eg: **int no1;**
  - To assign a value to the variable…..
  - <datatype> <name_of_variable>=<value>;
  - Eg: **int no1=133330; char c='A' String name="ABC"**
  - **Float pi=3.14 byte a=12;**

Standard Output

- To print to standard output use

  System.out.print(“expression “); // no newline

  System.out.println(expression); // newline

```
System.out.println( );  // just a newline
```

Class object method

Of language package

```
System.out.println( "x is: " + x + " y is: " + y );
```

Here x & y are variable ; and ***+*** *<u>used to concatenation</u>*

<u>NOTE: no comma (,) is used to separate the expression</u>

procedure to run 1st program

1) **Write** your First program in NOTEPAD(or any text editor).

2) **Save** the file using **<filename.java>** in **java\bin** folder.

{NOTE: filename must be name of class in program}

3) **Open** command prompt and call the directory

C:\Program Files (x86)\Java\jdk1.7.0\bin

4) **Compilation:** compile program using

<javac filename.java>

5) **Execution:** After successful compilation run the program using

<java filename.java>

The Simplest Java Application: Hello,World!

- Since Java is object-oriented, programs are organized into modules called classes, which may have data in variables and subroutines called methods.

```
import java.lang.*;   // default language package optional
class HelloWorld
{
    public static void main (String args[])
        {
        System.out.println("Hello World!");
        }
}
```

identifier

main() is the first method that is run.

public: access specifies

static: method belongs to only current class, no need to create object to access main()

void: does not return any value

String args[]: array of object(args[]) of class String

Standard Input

- To read the data from console at runtime

```
import java.util.*; // utility package
import java.lang.*;   // by default (optional…..)
 class Addition
{
  public static void main(String args[])
  {
        int x,y,z;
        System.out.println("Please Enter the two no.");
        Scanner s=new Scanner(System.in);
        x=s.nextInt();
        y=s.nextInt();
        z=x+y;
        System.out.print("Addition is=" +z);
}}
```

Object name

Constructor

Class in util package

Method in util package

Standard Input

- **Methods of Scanner Class:**
- **1) nextInt():-** For Integer Value.
- **2) nextFloat():-** For Floating point value.
- **3) nextDouble():-** For Double value.
- **4) next():-** for String Value etc.

Note: Class name should start with Capital Letter

eg: class Addition, class Stud123 etc….

Method(function) name should start with small letter.

eg: println(), nextInt(). Etc…

Alternative-1 Input

- To read the data from console at runtime

```
import java.io.*;    // utility package
import java.lang.*;  // by default (optional.....)
 class Addition
{
  public static void main(String args[]) throws Exception
  {
        float x,y,z;
        System.out.println("Please Enter the two no.");
        DataInputStream d=new DataInputStream(System.in);
        x=Integer.parseInt(d.readLine());
        y=Float.parseFloat(d.readLine());
        z=x+y;
        System.out.print("Addition is=" +z);
  }     }
```

compulsory

Object name

Class in io package

Method in io package

Alternative-1 Input

- **Methods of DataInputStream Class:**
- **1) parseInt():-** For Integer Value.
- **2) parseFloat():-** For Floating point value.
- **3) parseDouble():-** For Double value.
- **4) readLine():-** for String Value.

Alternative-2 Input

- To read the data from console at runtime

```
import java.io.*;   // utility package
import java.lang.*;  // by default (optional…..)
 class Addition
{
  public static void main(String args[]) throws Exception
  {
        System.out.println("Please Enter the two no.");
BufferedReader  b=new BufferedReader(new InputSreamReader(System.in));
        int x=Integer.parseInt(b.readLine());
        int y=Integer.parseInt(b.readLine());
        z=x+y;
        System.out.print("Addition is=" +z);
  }  }
```

Object name

Class in io package

Method in io package

JAVA TOKEN

- A smallest individual element in a program is called as token
- Java is collection of tokens, comments, separator, & white space.
- Java includes five different types of tokens.

1. Identifiers
2. Keywords
3. Literals
4. Operators
5. Separators

JAVA TOKEN

1. Identifier: *Identifier are the **name** used for variables, classes, methods, objects, labels, packages and interfaces in a program.*

- Name of all public methods and instance variables start with a lowercase letter.

  Eg: total, sum, display etc.

- When the name contains more than one word, the second and subsequent words are marked with leading uppercase characters.

  Ex: findAverage isLeapYear grandTotal

- All private and local variables use only lowercase letters combined with underscores.

  Ex: birth_day; total_avg_marks;

JAVA TOKEN

- All class and interface names starts with an uppercase letter and each subsequent letter of the word with uppercase letter.

  Ex: HelloJava, FirstClass, Example1 etc.

- Variables that represent constant values use all uppercase letters and underscores between words.

  Ex: MAX_MARKS, VALUE_OF_PI, GRADE etc.

JAVA TOKEN

2. Keywords: *Keywords are essential part of language definition. Keyword used with operator or separator according to the syntax.*

- *50+ Keyword available in JAVA*

abstract	Boolean	Break	byte	byvalue*
case	cast*	Catch	char	class
const	Continue	Default	do	double
else	Extends	False	final	finally
float	For	future*	generic*	goto*
if	Implement	Import	inner*	instanceof
int	Interface	Long	native	new
null	operator*	outer*	package	private
protected	Public	rest*	return	short
static	Super	Switch	synchronized	this
threadsafe*	Throw	Throws	transient	true
try	var*	Void	volatile	while

JAVA TOKEN

3. Literals: Literals are the sequence of characters (i.e. digits, letters and symbols) that represent constant values to be stored in variables.

- Java specifies five major types of literals.

i. Integer literals : int a=10;

ii. Real literals :double pi=3.14;

iii. Character literals :char ch='a';

iv. String literals : String str="hello";

v. Boolean literals :boolean value=true;

JAVA TOKEN

4. Operators: *An operator are the symbols that perform certain task on operands*.

List of Operators:

1. Arithmetic Operators
2. Assignment Operators
3. Increment / Decrement Operators
4. Relational Operators
5. Logical Operators
6. Conditional Operators
7. Special Operators
8. Bitwise Operator

JAVA TOKEN

1. **Arithmetic Operators:** *Arithmetic operators are used for basic arithmetic operation.*

   Consider: ***a=10 & b=3***

Operator	Meaning	Example	Output
+	Addition	a+b	
-	Subtraction	a-b	
*	Multiplication	a*a	
/	Division	a/b	
%	Modulus	a%b	

JAVA TOKEN

2. **Assignment Operators:** *Assignment operators are used to assign the value to the variable.*

Consider **A=5;**

Operator	Example	Output
=	A=5	5
+=	A+=5	
-=	A-=3	
*=	A*=3	
/=	A/=2	
%=	A%=3	

JAVA TOKEN

3. Increment / Decrement Operators: *These operators are used to increase & decrease the value by 1.*

Consider A=5

Operator	Meaning	Example	Output
++	**Increment by 1**	**A++**	
--	**Decrement by 1**	**A--**	

Two types of Increment & Decrement Operators:

- *Pre Increment/Decrement* & *Post Increment/Decrement:*

Consider **a=6 & b=0;**

Pre Increment	Post Increment
Firstly Input value is incremented by 1 and then assign input value to output variable **Ie: Both side value will be increment by 1**	**Firstly value of input variable is assign to the output variable, & then value of input variable is incremented by 1** **Ie: Only input variable is increment by 1**
E.g. b=++a **b= a=**	**E.g. b=a++** **b= a=**

JAVA TOKEN

Consider **a=6 & b=0;**

Pre Decrement	Post Decrement
Firstly Input value is decremented by 1 and then assign input value to output variable **Ie: Both side value will be decrement by 1**	**Firstly value of input variable is assign to the output variable, & then value of input variable is decremented by 1** **Ie: Only input variable is decrement by 1**
E.g. b=--a **b= a=**	**E.g. b=a--** **b= a=**

JAVA TOKEN

4. Relational Operators : *When we want to compare values of two variables for various means, relational operators are used. Consider:* **Let A=4, b=5,c=2**

Operator	Meaning	Example	Output
>	Greater than	A>c	True
<	Less than	A<b	
>=	Greater than equal to	A>=c	
<=	Less than equal to	A<=c	
==	Equal to	b==c	
!=	Not equal to	A!=c	

JAVA TOKEN

5. Logical Operators : *There are only three logical operators which are related with the logical decisions.*

Consider: **Let A=2, b=5,c=7**

Operator	Meaning	Example	Output
&&	AND (1,1=1 o/w 0)	(A>c) && (A<b)	False
||	OR (0,0=0 o/w 1)	(A>c) || (A<b)	
!	NOT (1=0 / 0=1)	!(A<=c)	

JAVA TOKEN

6. Conditional Operators : *also called as Ternary operator, used to check wheatear the condition is true or false*

Syntax

```
Variable_name=Condition ? Expreesion1 : Expression2;
```

Condition

true -> Expression 1

False -> Expression 2

```
int a=10;
int b= (5>3)?(a+10):(a-5);      o/p= 20
```

JAVA TOKEN

7. Special Operators: *Java supports two special operators. That is, instanceof and dot operator (.).*

a) Instanceof operator: *It returns true if the object on the left-hand side is an instance of the class given on the right-hand side.*

For example: maharashtra instanceof India

returns true if the object 'maharashtra' is the object of class 'India'. Otherwise it will return false.

b) Dot Operator (.): *It is used to access the instance variable and methods of the class using objects.*

Ex: company.salary(); *//reference to method salary*

company.employee; *//reference to variable employee*

JAVA TOKEN

8. Bitwise Operator: *In order to manipulate the data at the bit level, the bitwise operators are provided in Java*

Consider: A=12 (1100) & b=5 (?)

Operator	Meaning	Example	Output
&	AND (11=1 o/w 0)	A & b	4
|	OR (00=0, o/w 1)	A ! b	
~	Negation (1->0, 0->1)	~b	
^	Ex-or (01,10=1 o/w 0)	A^b	
>>	Signed Right Shift by 1	A>>1	
<<	Signed Left Shift by 1	b<<1	
>>>	Unsigned Right Shift by 1	b>>>1	

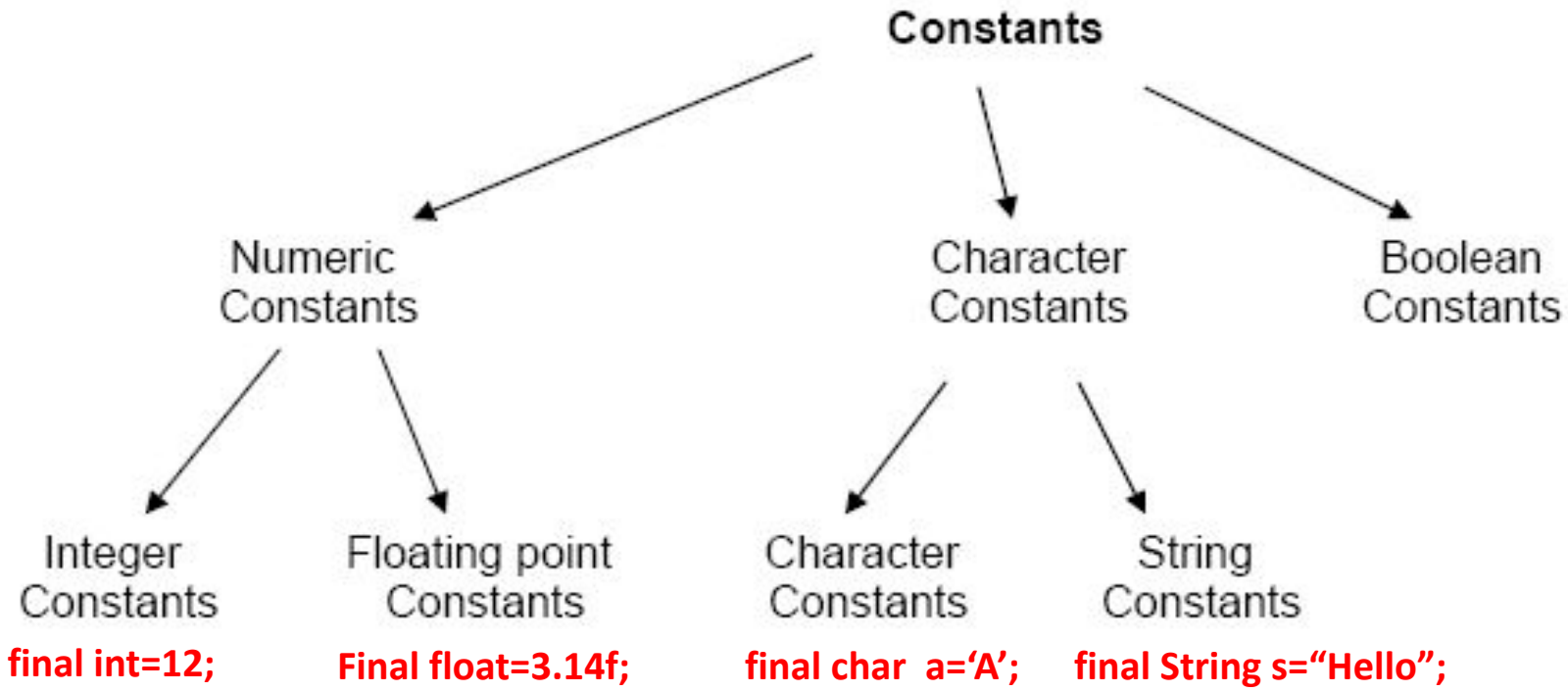
JAVA TOKEN

5. Separator: ***These are the symbols which are used to indicate where groups of codes are separated and arranged***

Separator	Name	Purpose
( )	Parenthesis	To contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Curly Braces	To contain the values of automatically initialized arrays. Also used to define a block of code for classes, methods, and local scopes.
[ ]	Square brackets	To declare array data types. Also used when using array values.
;	Semicolon	Terminating the statement
,	Comma	It separates the consecutive identifiers in a variable declaration and used to chain statements together inside a 'for' loop statement.
.	Dot/Period	To separate package names from sub-packages and classes. Also used to separate a variable or method from a reference variable.

Constant

- *Constants are the values which <u>do not change</u> during the execution of program.*
  - ***final*** *keyword used to declare constant value*
  - *Eg:* ***final int a=12;***



Escape Characters/Constants

Escape	Meaning
\n	New Line
\t	Horizontal Tab
\b	Back space
\’	Single Quote
\”	Double Quote
\\	Back slash

TO READ STRING & CHAR.

- Syntax

DataInputStream d=new DataInputStream(System.in);

- ***String str=d.readLine();*** *// for string*
- **char ch= (char) System.in.read();** // for character

- **Scanner s=new Scanner(System.in)**
- **String str=s.nextLine();**

Scope of variable

- **Scope determines the visibility of program elements** with respect to other program elements
- **Three type of Visibility….**
  1) **Instance variable**
  2) **Class variable**
  3) **local variable**

The instance & class variables are declared **inside the class**. They are created when the objects of the class are created and therefore they are associated with the objects.

Variables declared & used **inside a method** are called as local variables.

SCOPE OF VARIABLE

```
{                                   Block1

        int x = 0;

            {                       Block2
                -------;
                int m = 10;
            }

            {                       Block3
                -------;
                int y = 55;
            }

}
```

Fig. 1.19 Block scope of variables

Type conversation

- The process of converting one data type into another is called as CASTING.
- The process of assigning a Smaller value to Larger one is known as **Widening or promotion.**
  - **No loss of Information may cause.**
- The process of assigning a Larger value to Smaller one is known as **Narrowing or demotion.**
  - **Loss of Information may cause.**
- Eg1: byte b=75;

  int a=b;
  - (int is large to hold byte value so **no loss** happen)
- Eg2: int b=75;

  short a=b;
  - (short is small to hold int value so **loss** happen)

Type Casting

- The process of assigning a Larger value to Smaller one is known as **Narrowing or demotion.**
- Syntax: **<datatype> <varibale1> = <datatype> <varibale2>;**

-Eg: **int X=50;**

byte Y= (byte) X;

From	To
byte	short, char, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	Double

Table 1.15 type casting that results in no loss of information

Math Class

- Java support math functions through Math class defined in java.lang package
- All are static members
- Syntax:
  - Math.method_name();

  - Exmple:
    - **double A;**
    - **A=Math.ceil(12.60);**

Math is class found in lang package

Math function

Function	Meaning	Example
abs(a)	Return absolute value of a	a=-55 -> a=55.00
ceil(a)	Return Largest whole no >or = to a {ie: rounded foreword}	A=12.60 -> 13.00 A=-12.75 -> -12.00
floor(a)	Return Smallest whole no < or = to a {ie: rounded downword}	A=12.60 -> 12.00 A=-12.75 -> -13.00
round(a)	Return integer closest to the argument	a=12.51 -> 13 a=12.35 -> 12
pow(a,b)	Return a raised to b {ie: a^b}	a=4,b=2 -> 16.00
sqrt(a)	Return a square root of a	a=64 -> 8.00
exp(a)	Return e raised to a {ie: e^a}	a=1 -> 2.71
Min(a,b)	Return minimum value from a & b	a=10,b=12 -> 10.00
Max(a,b)	Return maximum value from a & b	a=10,b=12 -> 12.00

Decision making and Branching

- if
- if-else
- nested if-else
- ladder if-else
- Switch case

If Statement

Syntax:

```
if (Condition)
{
    // Statements for true;
}
    //Statements for true/false;
```

If Statement

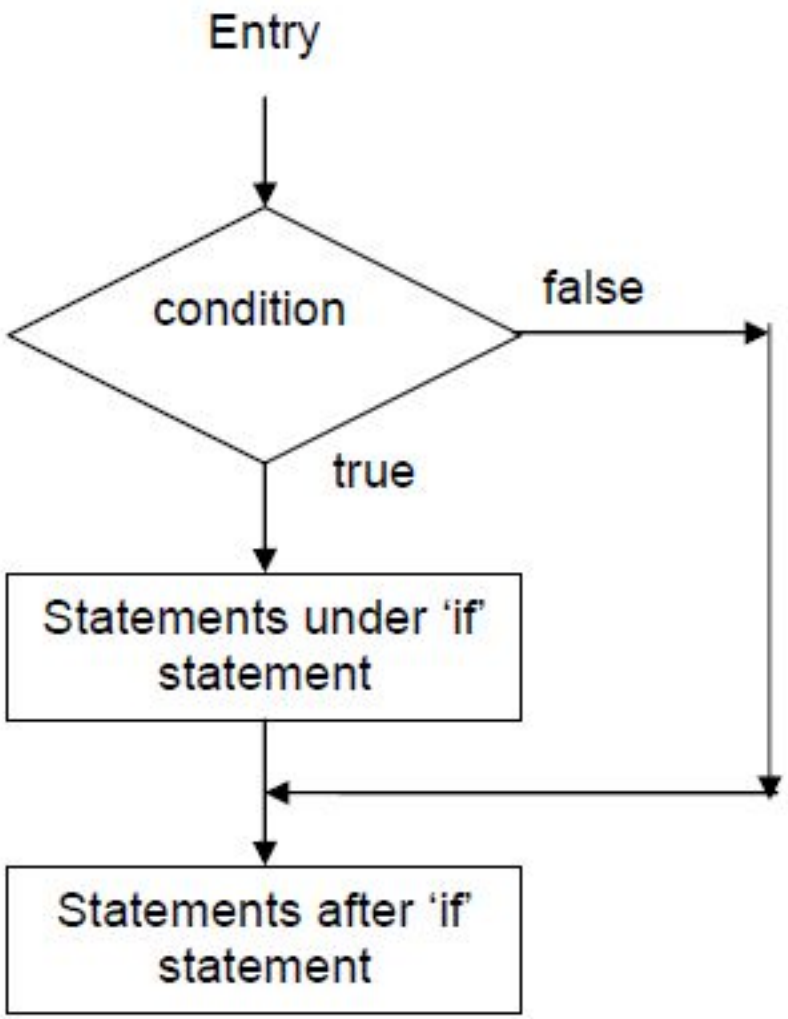


Fig. 1.20 Control flow of an 'if' statement

If-else Statement

Syntax:

```
if (Condition)
{
 // Statements for true;
}
else
{
 // Statements for false;
}
 // Statements for true/false;
```

If-else Statement

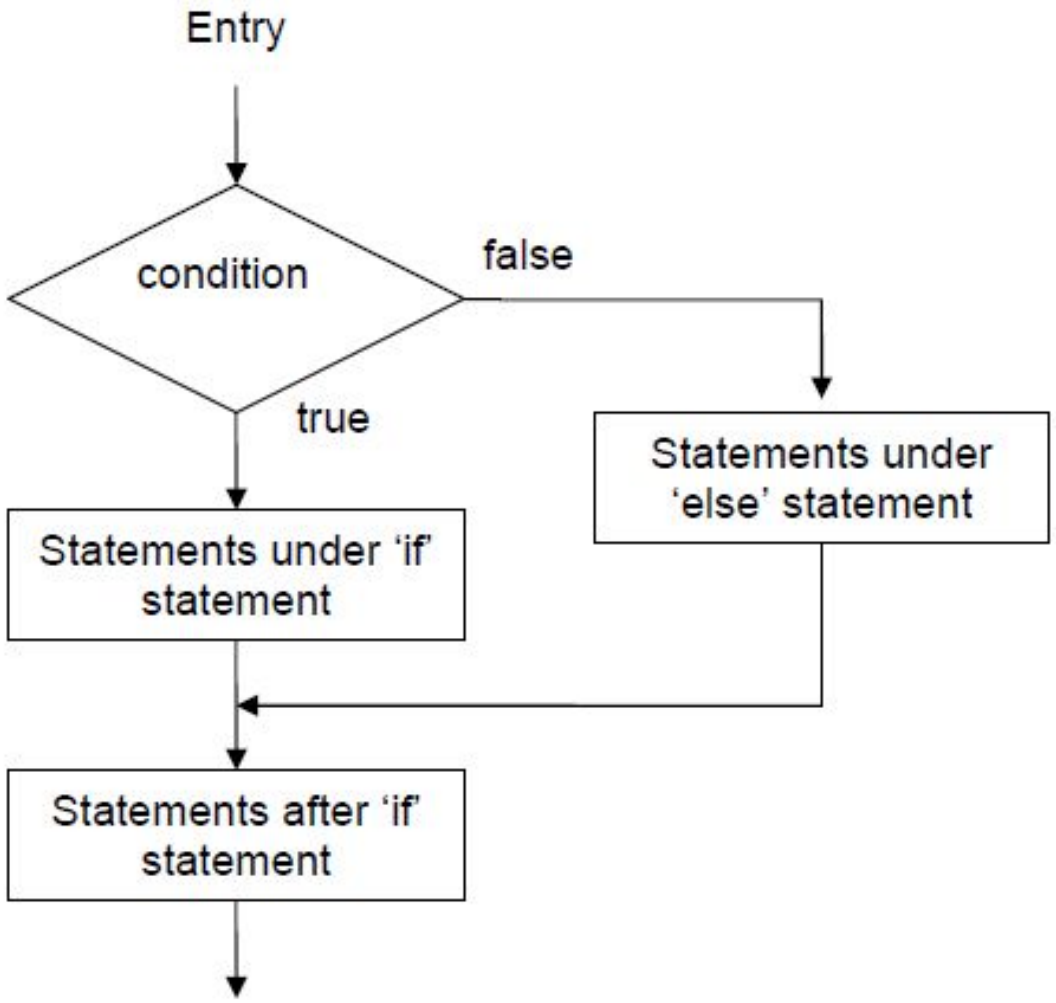


Fig. 1.21 Flowchart for if-else control structure

Nested if-else Statement

Syntax:

```
if (Condition1)
{
        // Statements for true;
    if(condition2)
    {
        // Statements for true;
    }
}
else
{
        // Statements for false;
    if(condition3)
    {
        // Statements for true;
    }
 }
    // Statements for true/false;
```

Ladder if-else Statement

Syntax:

```
if (Condition1)
{
    // condition 1 true
}
  else
   if(condition 2)
   {
      //condition one false & condition is 2 true
   }
     else
       if(condition 3)
       {
            // cond. 1, 2 is false & cond. 3 is true
       }
       :
       :
else
{
      //all condition false
}
```

Switch case

- Menu Driven Program base on user choice

- Syntax

```
        switch (Value/Expression)
  {
        case value1:
                //true statements
                 break;
        case value2:
                //true statements
                 break;
          :                                    :
        case value n:
                //true statements
                 break;
        default  :                                  //statements

}
```

Value may be
Integer / character / String/symbol

Switch case

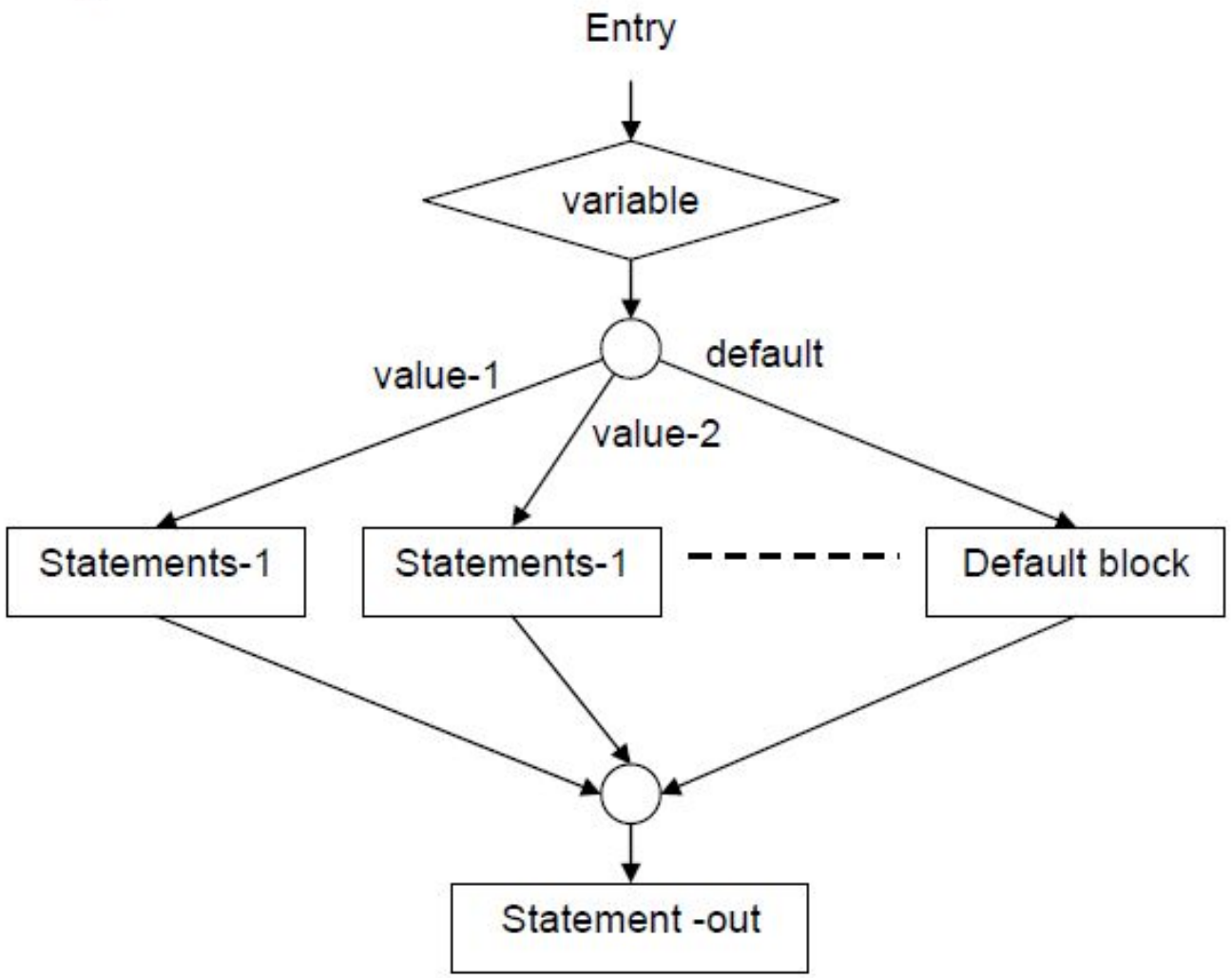


Fig. 1.23 Flow-chart for switch-case statement

Decision making and looping

- **while**
- **do-while**
- **for**

While loop

```
Initialization;
while(condition)
{
    // statement true

Increment/decrement operator;

}
```

While loop

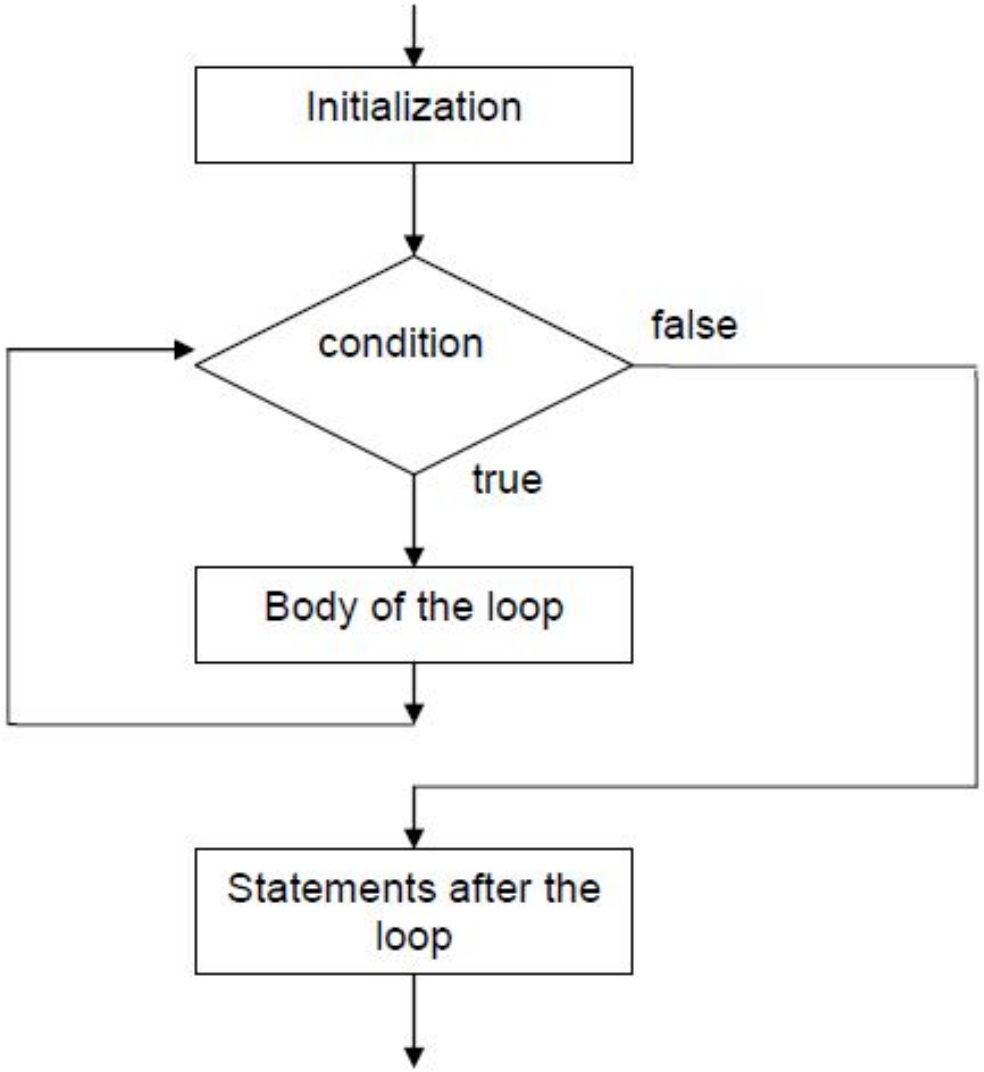


Fig. 1.24 Flowchart of while loop statement

Do-While loop

```
Initialization;
do
{
    // statement
Increment/decrement operator;

} while(condition);
```

Do-While loop

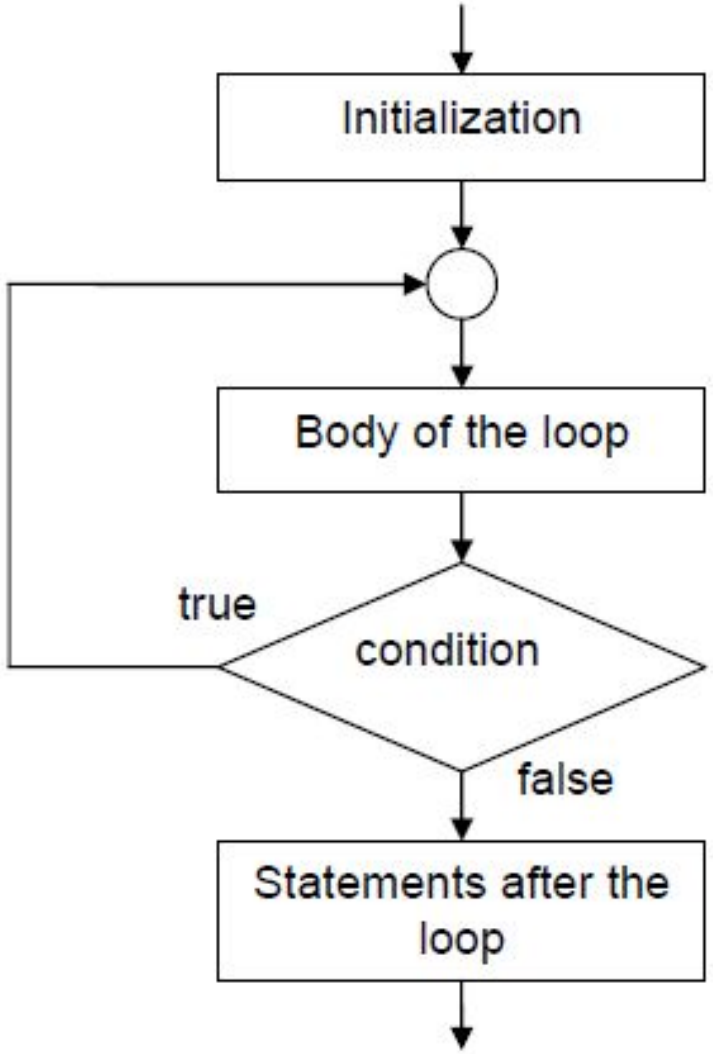
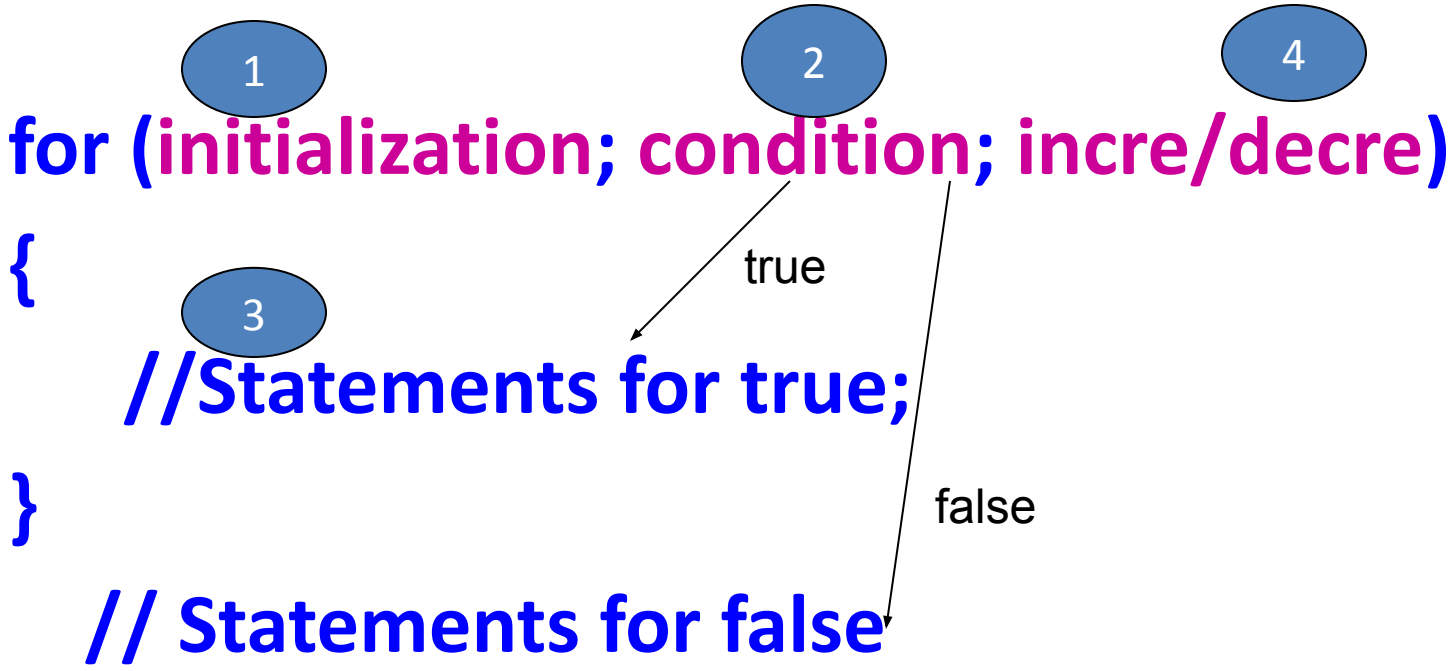


Fig. 1.25 Flowchart of do-while loop

For loop

- **syntax**



For loop

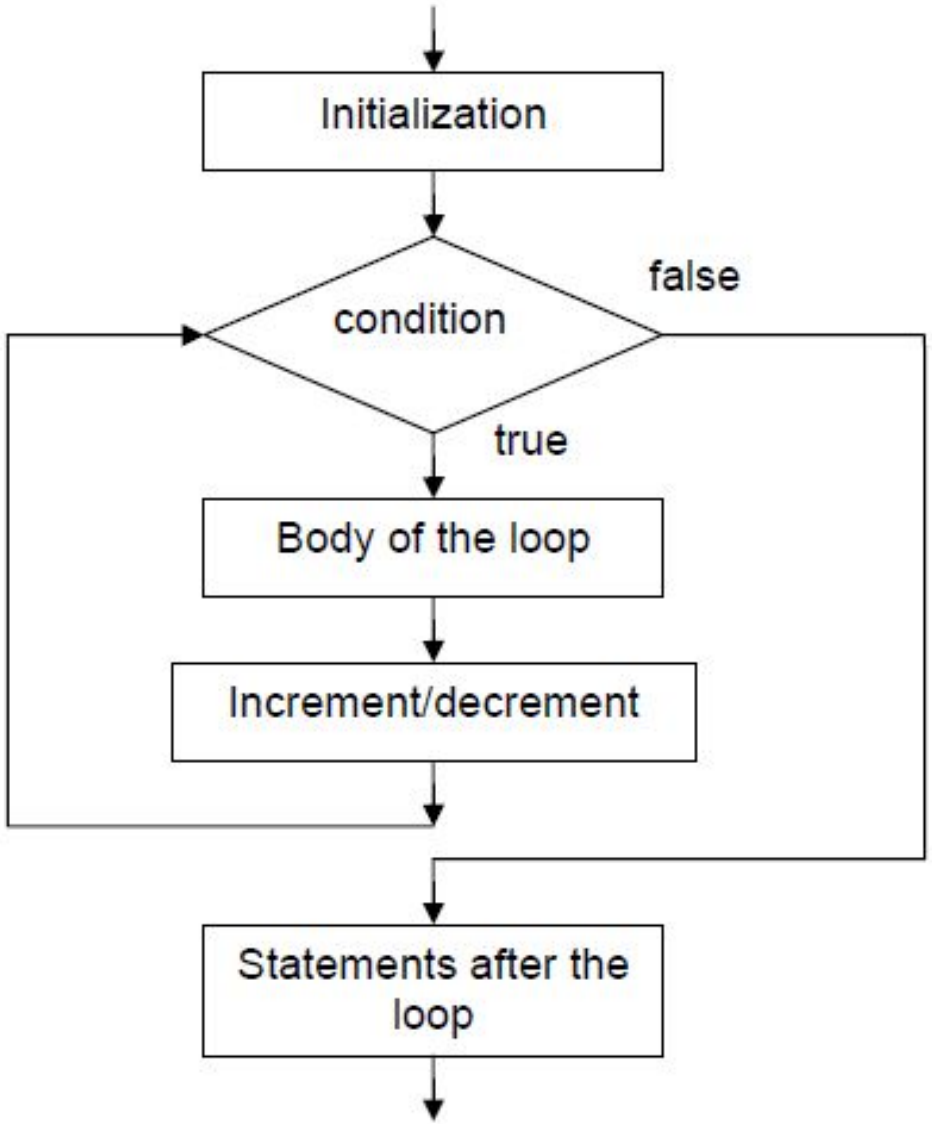


Fig. 1.26 Flowchart of the 'for' loop

Forms of FOR LOOP

- More than one initialization
  - for (x=10,y=0; y<10; y++)
- Inside scope of variable
  - for (int x=0; x<=10; x++)
- More than one condition
  - for (int x=10,y=0; y<10 && x>0; y++)
- More than one incre/decre
  - for (x=10, y=0; y<10; x--, y++)
- Only condition
  - for (; y<10 ;) // initialization above for loop & incre/decre inside body
- Empty for loop
  - for(; ;) // initialization above for loop & incre/decre inside body
    - Loop will Run infinitely…..

Jump statement

- Sometimes while executing the loop it is necessary to skip some part of the loop or exit from the loop when certain conditions are met. In these cases the jump statements are used.

Two types of jump statements:

- **break**
- **continue**

break statement

- **break :** It is used to *jump out of the loop* when this statement is encountered inside the loop.
- Generally, the '**break**' statement is associated with an 'if' statement. That is, when the condition is satisfied, the jump will occur.
- *break statement* directly get exited *from the current loop without considering the condition.*
- **Example:**

break statement

```
//demonstration of break statement
  class BreakDemo
  {
      public static void main(String args[])
      {
          int var = 10;
          while(var<20)
          {
              System.out.println(var);
              var = var + 1;
              if(var>15)
                  break;
          }
          System.out.println("Loop completed...");
      }
  }
```

Program 1.30 demonstration of break statement

OUTPUT: 10 11 12 13 14 15 Loop completed…

Break (go to) statement

Using break as a form of goto

```
class breakgoto
{
        public static void main(String args[])
        {
                int x;
                start:
                        for (x=0; x<10; x++)
                        {
                                System.out.println("x=" +x);

                                if(x==5)
                                        break start;

                        }

                        System.out.println("Loop completed...");
        }
}
```

continue statement

- Continue: When we want to continue the next iteration of the loop by skipping some part inside it, the ‘continue’ statement can be used.
- Continues doesn’t directly get terminate the current loop.
- The control get at next iteration by skipping the false condition

continue statement

```
// Demonstration of the 'continue' statement.
  class ContinueDemo
  {
      public static void main(String args[])
      {
        for(int i=0; i<10; i++)
        {
            if (i%2 == 0)
                continue;
            System.out.println(i + " ");
        }
      }
  }
```

Program 1.33 Demonstration of the 'continue' statement

OUTPUT: 1 3 5 7 9

END OF CHAPTER 1